INFORME DE LABORATORIO 05

INFORMACIÓN BÁSICA					
ASIGNATURA:	**INTRODUCCIÓN AL DESARROLLO WEB**				
TITULO DE LA PRÁCTICA:	*Entorno de Ejecución Node.js, Gestión de Paquetes con npm y Creación de Servidor HTTP con Módulos Nativos*				
NÚMERO DE PRÁCTICA:	*05*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*II*
FECHA DE PRESENTACIÓN	*19/10/2026*	**HORA DE PRESENTACIÓN**	*hh/mm/ss*		
INTEGRANTE (s) *Christopher William Vásquez Vasquez* *Torres Escalante Omar Ignacio*				**NOTA (0-20)**	
DOCENTE(s): **MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.**					

RESULTADOS Y PRUEBAS

1. EJERCICIO / PROBLEMA RESUELTO POR EL DOCENTE

Servidor HTTP básico resuelto utilizando módulos nativos:

```
// server.js const http = require('http');
const fs = require('fs');
const path = require('path');
const PORT = 3000; const server = http.createServer((req, res) => {
console.log(Petición recibida: ${req.method} ${req.url});
if (req.url === '/' && req.method === 'GET') {
const filePath = path.join(__dirname, 'public', 'index.html');
fs.readFile(filePath, (err, content) => {
if (err) {
res.writeHead(500, { 'Content-Type': 'text/plain' });
res.end('Error interno del servidor');
} else {
res.writeHead(200, {
'Content-Type': 'text/html;
charset=utf-8' });res.end(content);
}
});
} else if (req.url === '/api/status' && req.method === 'GET') {
res.writeHead(200, { 'Content-Type': 'application/json' });
res.end(JSON.stringify({ status: 'OK', uptime: process.uptime() }));
} else {
res.writeHead(404, { 'Content-Type': 'application/json' });
res.end(JSON.stringify({ message: 'Recurso no encontrado' })); } });
server.listen(PORT, () => {
console.log(`Servidor escuchando en http://localhost:${PORT}`); });
```

2. EJERCICIOS / PROBLEMAS PROPUESTOS

Caso Práctico: Servidor Backend Estático y API Básica de Consultas

Desarrollar una aplicación en Node.js desde cero implementando los siguientes requerimientos:

1. **Configuración de Proyecto npm:**
   - Inicializar el proyecto ejecutando npm init -y.
   - Configurar scripts en el package.json: "start": "node server.js" y "dev": "node --watch server.js".
   - Crear una estructura modular de carpetas: /public (archivos estáticos HTML/CSS), /data (archivo JSON con datos) y server.js.

2. **Construcción del Servidor HTTP Nactivo:**
   - Instanciar el servidor mediante http.createServer escuchando en el puerto 3000.
   - Servir archivos estáticos desde la carpeta /public asíncronamente con fs.readFile infiriendo correctamente el Content-Type (ej. text/html, text/css).

3. **Implementación de Endpoints Basados en Rutas:**
   - Ruta GET /api/estudiantes: Leer el archivo /data/estudiantes.json de forma asíncrona y retornar la lista de estudiantes en formato JSON con código 200 OK.
   - Ruta POST /api/estudiantes: Capturar los chunks de datos del body de la petición (evento req.on('data') y req.on('end')), parsear el objeto, agregarlo al archivo JSON de persistencia local y retornar el registro guardado con código 201 Created.
   - Manejo de rutas no existentes retornando un JSON con código 404 Not Found.

ESTRUCTURA / CODIGOS:

Data:

estudiantes.json

```
[
  {
    "id": 1,
    "nombre": "Omar Torres",
    "codigo": "20260001",
    "carrera": "Ingeniería de Sistemas"
  },
  {
    "id": 2,
    "nombre": "Christopher Vásquez",
    "codigo": "20260002",
    "carrera": "Ingeniería de Sistemas"
  }
]
```

Public:

app.js:

```
// Función para consultar el estado del servidor (GET /api/status)
async function obtenerStatus() {
    const contStatus = document.getElementById('resultado-status');
    try {
        const response = await fetch('/api/status');
        const data = await response.json();
        contStatus.textContent = JSON.stringify(data, null, 2);
    } catch (error) {
        contStatus.textContent = 'Error al consultar el estado del servidor.';
    }
}

// Función para consumir GET /api/estudiantes
async function obtenerEstudiantes() {
    const contGet = document.getElementById('resultado-get');
    try {
        const response = await fetch('/api/estudiantes');
        const data = await response.json();
        contGet.textContent = JSON.stringify(data, null, 2);
    } catch (error) {
        contGet.textContent = 'Error al conectar con el servidor.';
    }
}

// Función para consumir POST /api/estudiantes
async function registrarEstudiante() {
    const contPost = document.getElementById('resultado-post');

    const nuevoEstudiante = {
        nombre: document.getElementById('nombre').value,
        codigo: document.getElementById('codigo').value,
        carrera: document.getElementById('carrera').value
    };

    try {
        const response = await fetch('/api/estudiantes', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
```

```
        },
        body: JSON.stringify(nuevoEstudiante)
    });

    const resultado = await response.json();
    contPost.textContent = JSON.stringify(resultado, null, 2);

    // Si se registró con éxito (201 Created), actualizamos la lista automáticamente
    if (response.status === 201) {
        obtenerEstudiantes();
        document.getElementById('formEstudiante').reset();
    }
} catch (error) {
    contPost.textContent = 'Error al registrar el estudiante.';
}
}
```

index.html:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Laboratorio 05 - EPIS UNSA</title>
    <!-- CDN de Bootstrap 5 -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light py-4">

    <!-- Ampliamos el contenedor a max-width de 1140px para que las 2 columnas entren cómodas -->
    <div class="container" style="max-width: 1140px;">
        <header class="text-center mb-4">
            <h1 class="text-primary fw-bold">Escuela Profesional de Ingeniería de Sistemas</h1>
            <p class="lead text-secondary">Lab 05 - Caso Práctico: Servidor Backend Estático y API de
Consultas</p>
        </header>

        <!-- Fila 0: Estado del Servidor a lo ancho -->
        <div class="row mb-4">
            <div class="col-12">
                <div class="card shadow-sm">
                    <div class="card-header bg-info text-white fw-semibold">
                        Estado del Servidor (GET /api/status)
```

```
            </div>
            <div class="card-body">
              <button class="btn btn-info text-white mb-3" onclick="obtenerStatus()">
                Verificar Estado
              </button>
              <div class="bg-dark text-white p-3 rounded">
                <pre id="resultado-status" class="m-0"><code>Haz clic en el botón para consultar el uptime del servidor...</code></pre>
              </div>
            </div>
          </div>
        </div>
      </div>

      <!-- Fila 1: Dos columnas (Izquierda: GET Estudiantes | Derecha: POST Registrar) -->
      <div class="row g-4">
        <!-- Columna Izquierda: Consultar Estudiantes -->
        <div class="col-lg-6">
          <div class="card shadow-sm h-100">
            <div class="card-header bg-primary text-white fw-semibold">
              1. Consultar Estudiantes (GET /api/estudiantes)
            </div>
            <div class="card-body d-flex flex-column">
              <button class="btn btn-primary mb-3" onclick="obtenerEstudiantes()">
                Cargar Lista de Estudiantes
              </button>
              <div class="bg-dark text-white p-3 rounded flex-grow-1" style="max-height: 400px; overflow-y: auto;">
                <pre id="resultado-get" class="m-0"><code>Haz clic en el botón para consultar los datos...</code></pre>
              </div>
            </div>
          </div>
        </div>

        <!-- Columna Derecha: Registrar Estudiante -->
        <div class="col-lg-6">
          <div class="card shadow-sm h-100">
            <div class="card-header bg-success text-white fw-semibold">
              2. Registrar Nuevo Estudiante (POST /api/estudiantes)
            </div>
            <div class="card-body d-flex flex-column">
```

```
                <form id="formEstudiante" onsubmit="event.preventDefault(); registrarEstudiante();">
                    <div class="mb-3">
                        <label for="nombre" class="form-label">Nombre Completo</label>
                        <input type="text" class="form-control" id="nombre" required placeholder="Ej. Carlos Pérez">
                    </div>
                    <div class="mb-3">
                        <label for="codigo" class="form-label">Código</label>
                        <input type="text" class="form-control" id="codigo" required placeholder="Ej. 20260003">
                    </div>
                    <div class="mb-3">
                        <label for="carrera" class="form-label">Carrera</label>
                        <input type="text" class="form-control" id="carrera" required value="Ingeniería de Sistemas">
                    </div>
                    <button type="submit" class="btn btn-success w-100">
                        Guardar Estudiante
                    </button>
                </form>

                <div class="bg-dark text-white p-3 rounded mt-3">
                    <pre id="resultado-post" class="m-0"><code>El resultado del POST aparecerá aquí...</code></pre>
                </div>
            </div>
        </div>
    </div>
</div>

<script src="app.js"></script>
</body>
</html>
```

.gitignore:

```
node_modules/
```

package.json:

```
{
  "name": "lab05",
  "version": "1.0.0",
```

```
  "description": "Servidor HTTP nativo y API REST - Lab 05",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

server.js:

```
const http = require('http'); const fs = require('fs'); const path = require('path');
const PORT = 3000;
const server = http.createServer((req, res) => { console.log(Petición recibida: ${req.method} ${req.url});
// Requerimiento: Servir archivos estáticos del frontend (HTML, JS) desde la carpeta /public
if (req.method === 'GET' && (req.url === '/' || req.url.startsWith('/public/') || req.url.endsWith('.js') ||
req.url.endsWith('.html'))) {
  const targetPath = req.url === '/' ? 'index.html' : req.url;
  const filePath = path.join(__dirname, 'public', targetPath);

  const extname = path.extname(filePath);
  let contentType = 'text/html; charset=utf-8';
  if (extname === '.js') contentType = 'application/javascript; charset=utf-8';

  fs.readFile(filePath, (err, content) => {
    if (err) {
      res.writeHead(404, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ message: 'Archivo no encontrado' }));
    } else {
      res.writeHead(200, { 'Content-Type': contentType });
      res.end(content);
    }
  });

// Requerimiento: Endpoint de control de estado del servidor
} else if (req.url === '/api/status' && req.method === 'GET') {
  res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
  res.end(JSON.stringify({
    status: 'OK',
    uptime: process.uptime()
  }));

// Requerimiento: API GET para leer y retornar los registros del archivo JSON (estudiantes.json)
```

```
} else if (req.url === '/api/estudiantes' && req.method === 'GET') {
  const filePath = path.join(__dirname, 'data', 'estudiantes.json');

  fs.readFile(filePath, 'utf8', (err, data) => {
    if (err) {
      res.writeHead(500, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ message: 'Error al leer la base de datos' }));
    } else {
      res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
      res.end(data);
    }
  });

// Requerimiento: API POST para recibir datos, calcular ID autoincremental y persistirlos en el JSON
} else if (req.url === '/api/estudiantes' && req.method === 'POST') {
  let body = '';

  req.on('data', chunk => {
    body += chunk;
  });

  req.on('end', () => {
    try {
      const datosNuevos = JSON.parse(body);
      const filePath = path.join(__dirname, 'data', 'estudiantes.json');

      fs.readFile(filePath, 'utf8', (err, data) => {
        const estudiantes = err ? [] : JSON.parse(data);

        // Implementación de ID automático según la cantidad de elementos actuales
        const nuevoId = estudiantes.length + 1;

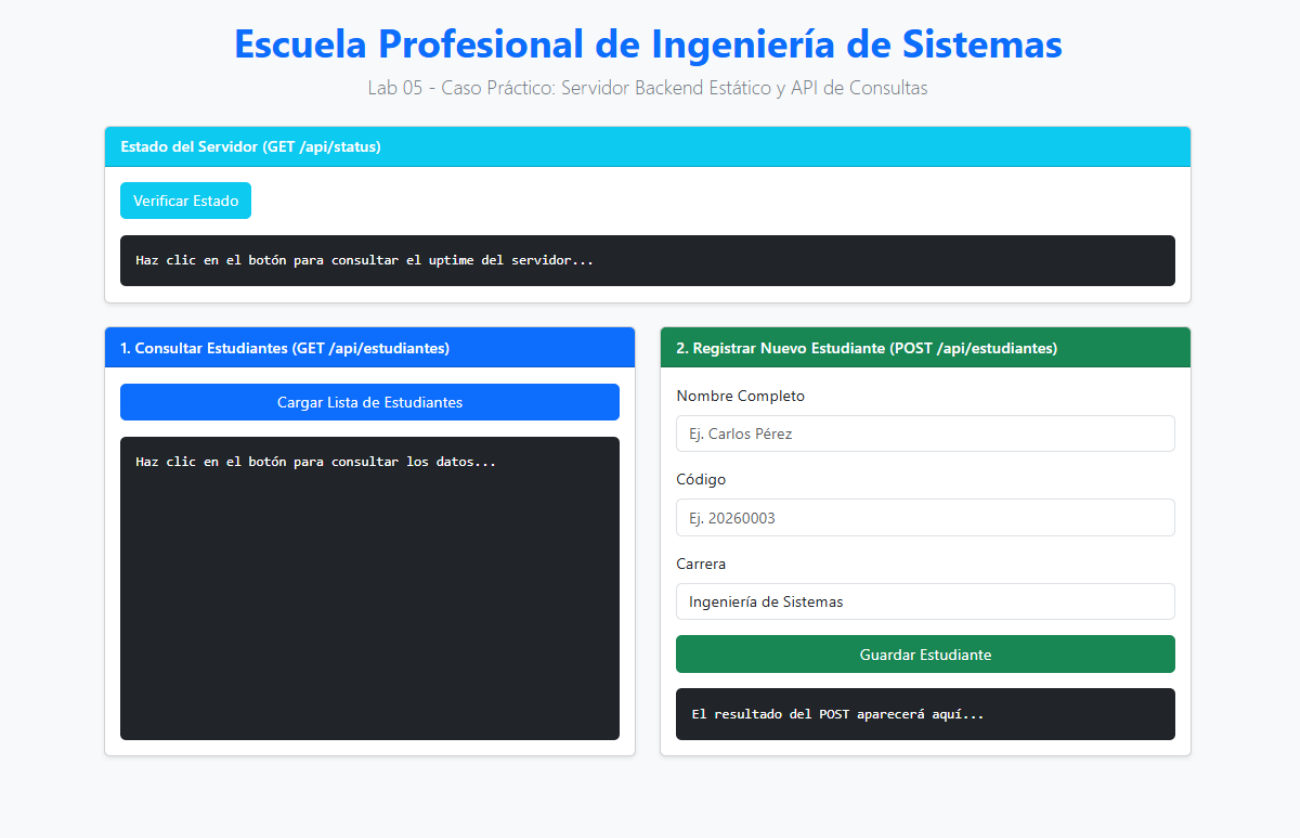
        const nuevoEstudiante = {
          id: nuevoId,
          nombre: datosNuevos.nombre,
          carrera: datosNuevos.carrera
        };

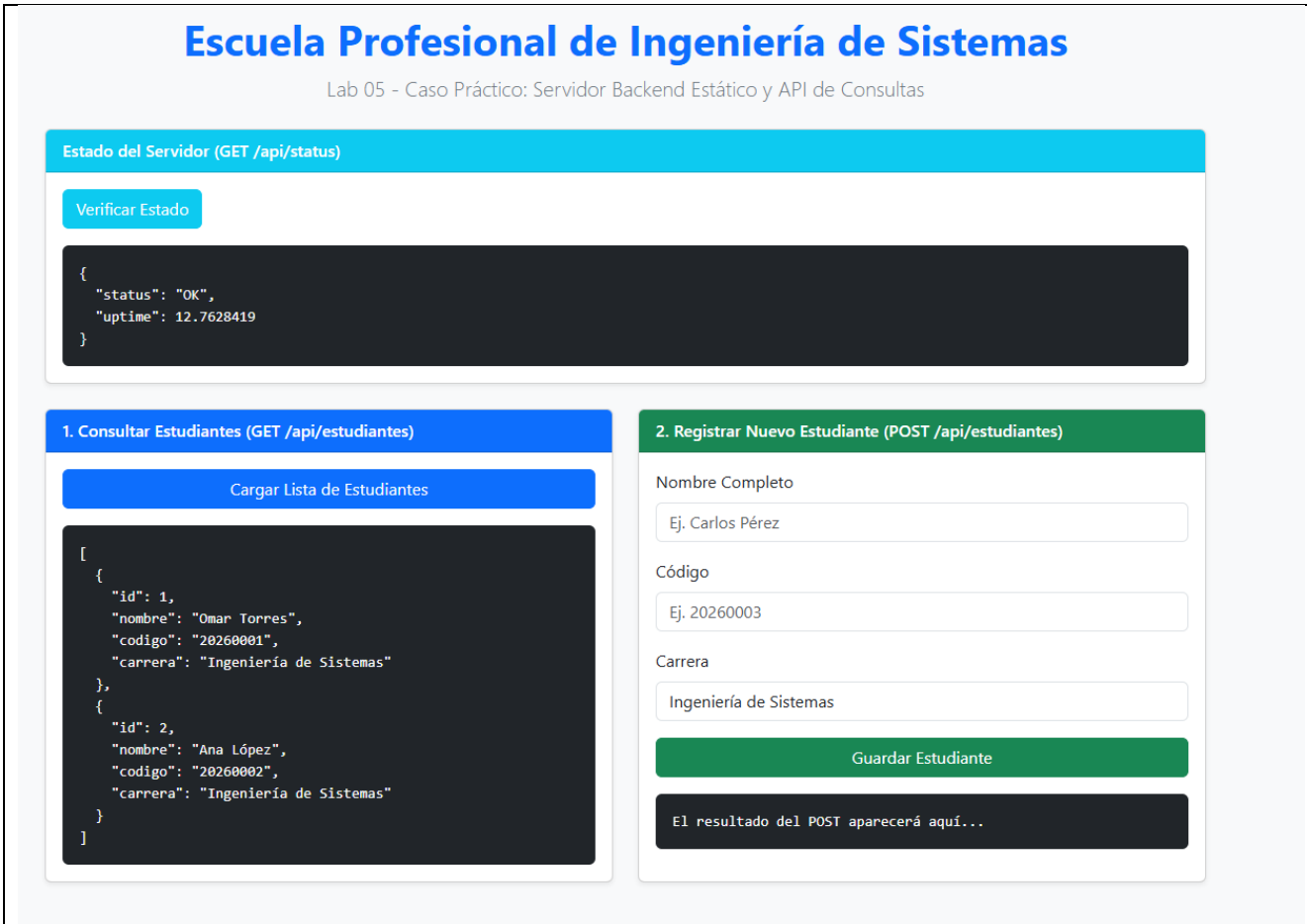
        estudiantes.push(nuevoEstudiante);

        fs.writeFile(filePath, JSON.stringify(estudiantes, null, 2), (err) => {
          if (err) {
            res.writeHead(500, { 'Content-Type': 'application/json' });
            res.end(JSON.stringify({ message: 'Error al guardar el estudiante' }));
          } else {
            res.writeHead(201, { 'Content-Type': 'application/json' });
```

```
                res.end(JSON.stringify({ message: 'Estudiante registrado con éxito', estudiante:
nuevoEstudiante }));
              }
            });
          });
      } catch (e) {
          res.writeHead(400, { 'Content-Type': 'application/json' });
          res.end(JSON.stringify({ message: 'JSON mal formado' }));
      }
   });

// Requerimiento: Manejo estándar de rutas no encontradas (404)
} else {
   res.writeHead(404, { 'Content-Type': 'application/json; charset=utf-8' });
   res.end(JSON.stringify({ message: 'Recurso no encontrado' }));
}
});
server.listen(PORT, () => { console.log(Servidor escuchando en http://localhost:${PORT}); });
```





Laboratorio: ***https://github.com/otorrese-dev/IDWeb/tree/main/Lab05***

3. CUESTIONARIO

1. **Explique la arquitectura orientada a eventos y el modelo de I/O No Bloqueante (Non-blocking I/O) de Node.js. ¿En qué se diferencia de servidores multihilo tradicionales como Apache Tomcat?**

   Node.js ejecuta tu código JavaScript en un único hilo y delega las operaciones lentas (leer archivos, red, base de datos) al sistema operativo o a un pool interno de hilos. Mientras esas operaciones se completan, el hilo principal sigue atendiendo otras peticiones. Cuando una operación termina, se encola un evento con su callback, y el **Event Loop** lo ejecuta en cuanto el hilo está libre. Esto es I/O no bloqueante. En servidores multihilo tradicionales como Apache Tomcat, cada petición suele ocuparse con un hilo propio que queda bloqueado esperando la I/O. Eso consume más memoria (cada hilo tiene su pila) y limita la cantidad de conexiones simultáneas.

2. **¿Qué función cumple la carpeta node_modules y por qué nunca debe incluirse en el control de versiones (Git)? ¿Qué archivo controla exactamente el árbol de dependencias?**

   node_modules es la carpeta donde npm descarga e instala las dependencias del proyecto y las de esas dependencias. No debe subirse a Git porque suele pesar cientos de MB, contiene miles de archivos que cualquiera puede regenerar con npm install, y puede tener binarios específicos del sistema operativo. Por eso se agrega al .gitignore. El archivo package.json declara las dependencias y sus rangos de versión, y package-lock.json fija el árbol exacto de versiones instaladas, de modo que todos obtienen lo mismo. Ambos sí se versionan.

3. **¿Cuál es la diferencia entre utilizar métodos síncronos de File System (fs.readFileSync) frente a métodos asíncronos (fs.readFile) en un servidor de producción?**
   readFileSync bloquea el hilo principal hasta terminar de leer el archivo, y durante ese tiempo el servidor no atiende a nadie más. readFile es asíncrono: delega la lectura, el hilo queda libre y el resultado llega después por un callback (o promesa). En producción, con muchos usuarios, usar métodos síncronos en el manejo de peticiones hace que cada lectura de disco retrase a todos los demás, y degrada el rendimiento.

CONCLUSIONES

Tras la elaboración del laboratorio se pudo identificar que Node.js permite usar JavaScript también en el servidor, y los módulos nativos bastan para construir un servidor funcional sin librerías externas. Además que usar métodos asíncronos como fs.readFile mantiene libre el Event Loop y permite atender varias peticiones al mismo tiempo.
Parsear el cuerpo del POST por chunks muestra cómo llegan realmente los datos por la red, algo que frameworks como Express esconden. Y por último package.json, sus scripts y el .gitignore dejan el proyecto reproducible en cualquier equipo con npm install.

METODOLOGÍA DE TRABAJO

1. ***Análisis**: se revisaron Node.js, el Event Loop, npm y los módulos nativos http, fs y path.*
2. ***Configuración**: se inicializó el proyecto con npm init -y, se definieron los scripts start y dev y se organizaron las carpetas public, data y server.js.*
3. ***Implementación:** se construyó el servidor con http.createServer, la entrega de estáticos con fs.readFile y path.join, y los endpoints GET y POST (capturando el cuerpo por chunks) con respuestas JSON y códigos 200, 201, 400 y 404.*
4. ***Pruebas:** se verificaron todas las rutas con curl o Postman, incluyendo casos de error.*